

Network Flow

COMP4128 Programming Challenges

School of Computer Science and Engineering
UNSW Sydney

Term 2, 2026

Network Flow

Flow Networks

Maximum
Flow

Flow
Algorithms

Ford-
Fulkerson

Edmonds-
Karp

Dinic

Faster
Algorithms

Bipartite
Matching

Min Cut

Related
Problems

Example
Problems

- 1 Flow Networks
- 2 Maximum Flow
- 3 Flow Algorithms
 - Ford-Fulkerson
 - Edmonds-Karp
 - Dinic
 - Faster Algorithms
- 4 Bipartite Matching
- 5 Min Cut
- 6 Related Problems
- 7 Example Problems

Network Flow

Flow Networks

Maximum
Flow

Flow
Algorithms

Ford-
Fulkerson

Edmonds-
Karp

Dinic

Faster
Algorithms

Bipartite
Matching

Min Cut

Related
Problems

Example
Problems

- A *flow network*, or a *flow graph*, is a directed graph where each edge has a *capacity* that *flow* can be pushed through.
- There are (usually) two distinguished vertices, called the source (s) and the sink (t) that the flow comes from and the flow goes to.
- Intuitively, flow graphs can be likened to networks of pipes, each with a limit on the volume of water that can flow through it per unit of time.
- The source has an infinite supply of water, the sink can drain an infinite amount of water. But for every other node, water in = water out. For every edge, water in $\leq$ capacity.
- Formally defined as a system of inequalities, involving theoretical terms like *skew symmetry* and *flow conservation*. Not necessary for our purposes.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

1 Flow Networks

2 Maximum Flow

3 Flow Algorithms

- Ford-Fulkerson
- Edmonds-Karp
- Dinic
- Faster Algorithms

4 Bipartite Matching

5 Min Cut

6 Related Problems

7 Example Problems

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- The maximum flow problem is to find, given a flow graph with its edge capacities, what the maximum flow from the source to the sink is.
- We restrict ourselves to integer capacities. This subclass is already interesting enough. There are subtle traps with real capacities. In particular, DON'T use Ford-Fulkerson with real flows.
- The *integrality theorem* states that if all the edges in the graph have integer capacities, then there exists a maximum flow where the flow through every edge is an integer.
 - This doesn't mean that you can't find a maximum flow where the flow in some edges *isn't* integer, only that you won't need to.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

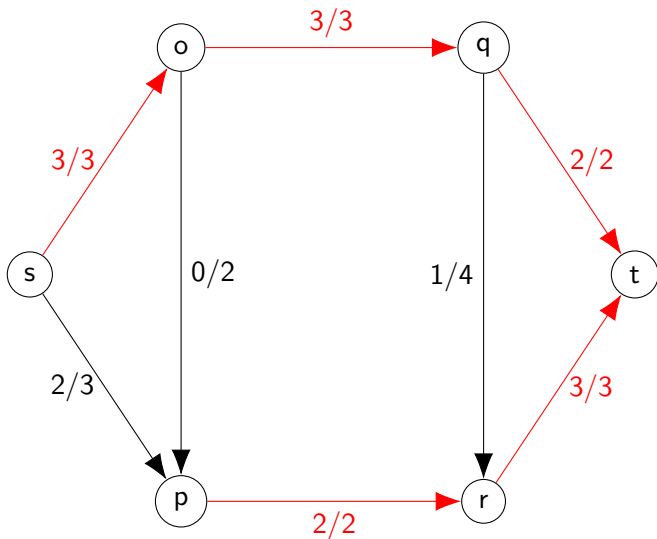
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- The dual of max flow.
- Given a graph with a source, a sink and edge weights, find the set of edges with the smallest sum of weights that needs to be removed to disconnect the source from the sink.
- In a directed graph, define a s - t cut to be a partition of the vertices into two sets, one containing the source s , the other containing the sink t . The capacity of such a cut is defined to be the sum of capacities of edges going from the source partition to the sink partition. The min cut is the minimum capacity of all s - t cuts.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- It turns out that these two problems are actually equivalent!
- The max-flow/min-cut theorem states that the value of the minimum cut, if we set edge weights to be capacities in a flow graph, is the same as the value of the maximum flow.
- We restrict ourselves to integer weights for same reason as with flow.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- To see why this is so, we examine a modification of the original flow graph called the residual graph.
- The residual graph of a flow graph has all the same edges as the original graph, as well as a new edge going in the other direction for each original edge, with capacity zero.
- Flowing along our original edges is like adding water to a pipe. Flowing along a residual edge does the opposite, it removes water from the pipe.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- We define an *augmenting path* as a path from s to t as a path that travels only on edges with capacity strictly greater than current flow.
- Clearly, if we can find an augmenting path in our graph, we can increase the total flow of the graph.
- If we keep finding augmenting paths, and decreasing the capacities of the edges they pass through, we can keep increasing the flow in our graph.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Whenever we flow along an edge, we should also do the opposite on the residual edge. So if we increase flow by 1, we should either decrease flow by 1 on the residual or, equivalently, increase capacity by 1. In this way, the sum of slacks of both edges remains constant.
- Intuitively, this is a mechanism allowing us to modify our already chosen augmenting paths. Since we keep the sum of slacks of both edges constant, flowing using the residual edge actually removes flow from the original edge. So in our augmenting path, it represents replacing the flow in the original edge with a new source of flow.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- It can be seen that if we can't find an augmenting path in our flow graph, then we have a maximum flow.
- Furthermore, this implies that in our residual graph, there exists no path from s to t , so we have discovered a cut.
- This cut must be a minimum cut, because if there were another smaller cut, it would be impossible to push as much flow as we have already pushed.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- So far, we have almost always represented graphs using only an adjacency list.
- We do need to iterate over the edges adjacent to v in $O(\deg v)$.
- But if you flow the edge $u \rightarrow v$, you also have to update the edge $v \rightarrow u$.
- Unweighted adjacency list, weights in adjacency matrix.
- Flow algorithms have higher complexity so we are almost always running out of time before we run out of memory.

Network Flow

Flow Networks

Maximum
Flow

Flow
Algorithms

Ford-
Fulkerson

Edmonds-
Karp

Dinic

Faster
Algorithms

Bipartite
Matching

Min Cut

Related
Problems

Example
Problems

```
struct FlowNetwork {
    int n;
    vector<vector<ll>> adjMat;
    vector<vector<int>> adjList;

    FlowNetwork (int _n): n(_n) {
        // adjacency matrix is zero-initialised
        adjMat.resize(n);
        for (int i = 0; i < n; i++) { adjMat[i].resize(n); }
        adjList.resize(n);
    }

    void add_edge (int u, int v, ll c) {
        // add to any existing edge without overwriting
        adjMat[u][v] += c;
        adjList[u].push_back(v);
        adjList[v].push_back(u);
    }

    void flow_edge (int u, int v, ll c) {
        adjMat[u][v] -= c;
        adjMat[v][u] += c;
    }
};
```

Network Flow

Flow Networks

Maximum
Flow

Flow
Algorithms

Ford-
Fulkerson

Edmonds-
Karp

Dinic

Faster
Algorithms

Bipartite
Matching

Min Cut

Related
Problems

Example
Problems

- 1 Flow Networks
- 2 Maximum Flow
- 3 Flow Algorithms**
 - Ford-Fulkerson
 - Edmonds-Karp
 - Dinic
 - Faster Algorithms
- 4 Bipartite Matching
- 5 Min Cut
- 6 Related Problems
- 7 Example Problems

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- The previously described augmenting paths algorithm for finding maximum flows is called the Ford-Fulkerson algorithm.
- As a summary, it just keeps finding augmenting paths and pushing flow through them until there are none left.
- Each graph search takes $O(E)$ time to send a single unit of flow.
- Therefore the time complexity is $O(Ef)$ where f is the maximum flow.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

```
// add to FlowNetwork struct
bool augment(int u, int t, ll f, vector<bool>& seen) {
    if (u == t) { return true; }
    if (seen[u]) { return false; }
    seen[u] = true;
    for (int v : adjList[u]) {
        if (adjMat[u][v] >= f && augment(v,t,f,seen)) {
            flow_edge(u,v,f);
            return true;
        }
    }
    return false;
}

ll ford_fulkerson(int s, int t) {
    ll res = 0;
    vector<bool> seen(n, false);
    while (augment(s, t, 1, seen)) {
        res++;
        fill(seen.begin(), seen.end(), false);
    }
    return res;
}
```

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- The f in the time complexity of Ford-Fulkerson is not ideal, because f could be exponential in the size of the input.
- It turns out that if you always take the shortest augmenting path, instead of any augmenting path, and increase the flow by the minimum capacity edge on your augmenting path, you need to find at most $O(VE)$ augmenting paths total.
- This gives a total time complexity of $O(\min(VE^2, Ef))$.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Proof (Sketch): Let G_f be the residual network after enacting flow f (so only leave edges that have more capacity than flow). Let $d_i(v)$ be the unweighted shortest distance from source s to v in G_f^i (the residual graph after step i of Edmonds-Karp).
- The main claim is that $d_i(v) \leq d_{i+1}(v)$, i.e: that Edmonds-Karp only increases distances from the source.
- You can roughly convince yourself of this by looking at what a step of Edmonds-Karp does to G_f^i .

Network Flow

Flow Networks

Maximum
FlowFlow
AlgorithmsFord-
FulkersonEdmonds-
Karp

Dinic

Faster
AlgorithmsBipartite
Matching

Min Cut

Related
ProblemsExample
Problems

- Each step of Edmonds-Karp runs BFS, and flows the edges along the shortest $s \rightarrow t$ path.
- This saturates some edges of G_f^i , removing them from G_f^{i+1} . It may also introduce some residue edges into G_f^{i+1} .
- But removing edges doesn't help decrease distances (obvious).
- And if $v \rightarrow u$ is a new residue edge, then v must have been further from s than u was. Thus the new residue edge goes 'backwards' relative to a BFS, so it too won't reduce distances.
 - Rigorous proof requires induction.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Thinking of the BFS orientation of edges also helps to prove our actual main claim: that an edge can only be saturated at most $n/2$ times.
- Say an edge $e : u \rightarrow v$ is saturated in iteration i , with u being d distance from the source. For e to be unsaturated, there must be some flow through its residue.
- But since distances don't decrease, for the residue to appear in the BFS tree, at some point u 's distance needs to be 1 more than v 's.
- But v 's distance is already $d + 1$, so for this to happen u 's distance must be $d + 2$. Hence the next time e is saturated, $d(u)$ has increased by 2. This can occur at most $n/2$ times.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Finally, each round of Edmonds-Karp saturates at least one edge by design. So there can be at most $O(VE)$ rounds. Each is a BFS ($O(E)$), hence running time is $O(VE^2)$.

Network Flow

Flow Networks

Maximum
FlowFlow
AlgorithmsFord-
FulkersonEdmonds-
Karp

Dinic

Faster
AlgorithmsBipartite
Matching

Min Cut

Related
ProblemsExample
Problems

```

// add to FlowNetwork struct, replacing Ford-Fulkerson code
ll augment(int s, int t) {
    queue<int> q; // BFS for path with fewest edges
    q.push(s);
    // pred[v] = vertex from which v was found in BFS
    vector<int> pred(n,-1);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adjList[u]) {
            if (adjMat[u][v] <= 0) { continue; }
            if (pred[v] == -1) {
                pred[v] = u;
                q.push(v);
            }
        }
    }
    if (pred[t] == -1) { return 0; } // can't reach the sink
    ll res = INF; // must be bigger than the largest edge capacity
    // walk over the path backwards to find the minimum edge
    for (int v = t; v != s; v = pred[v]) { res = min(res, adjMat[pred[v]][v]); }
    // do it again to adjust capacities
    for (int v = t; v != s; v = pred[v]) { flow_edge(pred[v],v,res); }
    return res;
}

```

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

```
ll edmonds_karp(int s, int t) {  
    ll res = 0;  
    while (ll x = augment(s,t)) { res += x; }  
    return res;  
}
```


Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- The most useful one for contests.
- Essentially just a more optimized implementation of Edmonds-Karp.
- The basic idea is the exact same, we repeatedly augment along the shortest augmenting path.
- However, we work in phases. In each phase, we augment until the shortest distance from source to sink has increased. Hence there are $O(V)$ phases.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- The key is to make the BFS orientation part of the earlier proof more explicit.
- Definition: the *level graph* is the graph restricted to only edges (u, v) where $d[u] + 1 = d[v]$.
- The speedup comes from realising that the level graph only has to be rebuilt at the start of each phase. This allows us to find each augmenting path in $O(V)$ amortized.
- Each phase has $O(E)$ augmenting paths. Hence, we get $O(V^2 E)$ overall.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- At the start of each phase, construct the level graph. Ignore all edges not in it.
- The main claim is: once there is no flow in the level graph, the distance from the source to sink has increased. The same ideas as the earlier proof sketch work here too.
- Now that we are working on the level graph, any path we find is a shortest augmenting path. So we can use a DFS to find these.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- The **key** is, say we are at node u . If we DFS to child c of node u and there are no augmenting paths from c to the sink, then child c is useless and we delete child c from node u .
- So our DFS will always find an augmenting path by trying the very first child of each node, except $O(E)$ times when we mess up. Amortized out, this means we get $O(V)$ per augmenting path, with an extra **overall** cost of $O(E)$ for the times we have to delete a child.
- Hence each phase is $O(EV)$, there are at most $O(E)$ augmenting paths and each is found in $O(V)$ (the extra $O(E)$ is dwarfed since it is $O(E)$ **per** phase).
- Overall complexity: $O(\min(V^2E, Ef))$.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- As an implementation note, we don't actually delete children.
- We instead use our adjacency list as our children array.
- For each node, we store the index of the first child that we have not "deleted".

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

```
struct FlowNetwork {
    int n;
    vector<vector<ll>> adjMat, adjList;
    // level[v] stores dist from s to v
    // uptochild[v] stores first non-useless child.
    vector<int> level, uptochild;

    FlowNetwork (int _n): n(_n) {
        // adjacency matrix is zero-initialised
        adjMat.resize(n);
        for (int i = 0; i < n; i++) { adjMat[i].resize(n); }
        adjList.resize(n);
        level.resize(n);
        uptochild.resize(n);
    }

    void add_edge (int u, int v, ll c) {
        // add to any existing edge without overwriting
        adjMat[u][v] += c;
        adjList[u].push_back(v);
        adjList[v].push_back(u);
    }

    void flow_edge (int u, int v, ll c) {
        adjMat[u][v] -= c;
        adjMat[v][u] += c;
    }
}
```

Network Flow

Flow Networks

Maximum

Flow

Flow

Algorithms

Ford-

Fulkerson

Edmonds-

Karp

Dinic

Faster

Algorithms

Bipartite

Matching

Min Cut

Related

Problems

Example

Problems

```
// constructs the level graph and returns whether the sink is still
// reachable
bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    queue<int> q;
    q.push(s);
    level[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        uptochild[u] = 0; // reset uptochild
        for (int v : adjList[u]) {
            if (level[v] == -1 && adjMat[u][v] > 0) { // unseen and has
                // capacity
                level[v] = level[u] + 1;
                q.push(v);
            }
        }
    }
    return level[t] != -1;
}
```

Network Flow

Flow Networks

Maximum

Flow

Flow

Algorithms

Ford-

Fulkerson

Edmonds-

Karp

Dinic

Faster

Algorithms

Bipartite

Matching

Min Cut

Related

Problems

Example

Problems

```
// finds an augmenting path with up to f flow.
ll augment(int u, int t, ll f) {
    if (u == t) { return f; } // base case.
    // note the reference here! we increment uptochild[u], i.e. walk through
    // u's neighbours
    // until we find a child that we can flow through
    for (int &i = uptochild[u]; i < adjList[u].size(); i++) {
        int v = adjList[u][i];
        // use only edges in the level graph with remaining capacity
        if (level[v] == level[u] + 1 && adjMat[u][v] > 0) {
            // revised flow is constrained also by this edge
            ll rf = augment(v,t,min(f, adjMat[u][v]));
            // found a child we can flow through!
            if (rf > 0) {
                flow_edge(u,v,rf);
                return rf;
            }
        }
    }
    level[u] = -1;
    return 0;
}
```


Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

```
||| 11 dinic(int s, int t) {  
|||     11 res = 0;  
|||     while (bfs(s,t)) {  
|||         while (11 x = augment(s, t, INF)) { res += x; }  
|||         // when no further paths, find new level graph  
|||     }  
|||     return res;  
||| }  
||| };
```

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Augmenting path-based algorithms tend to run much faster in practice than their worst case time complexity suggests.
 - In particular, Dinic's pretty much never behaves anything close to the worst case.
 - Kind of annoying for competitions.
 - If the graph has any structure, can probably run $V^2 E \approx 10^{12}$ in less than a second.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Faster flow algorithms exist, but they don't see much use in programming competitions.
- Many flow algorithms. Some of interest:
 - Capacity scaling $O(E^2 \log C)$ where C is the max capacity of an edge
 - Highest Label Preflow Push $O(V^2 \sqrt{E})$.
- Recently, it was shown that this problem can be solved in “almost linear” time.

Network Flow

Flow Networks

Maximum
Flow

Flow
Algorithms

Ford-
Fulkerson

Edmonds-
Karp

Dinic

Faster
Algorithms

Bipartite
Matching

Min Cut

Related
Problems

Example
Problems

- 1 Flow Networks
- 2 Maximum Flow
- 3 Flow Algorithms
 - Ford-Fulkerson
 - Edmonds-Karp
 - Dinic
 - Faster Algorithms
- 4 Bipartite Matching**
- 5 Min Cut
- 6 Related Problems
- 7 Example Problems

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

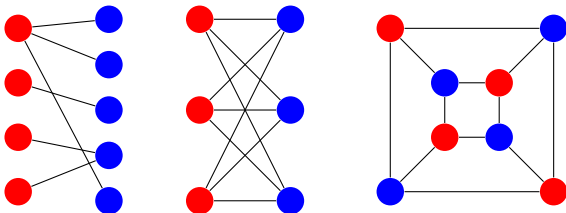
Bipartite Matching

Min Cut

Related Problems

Example Problems

- A bipartite graph is one where the vertices can be partitioned into two sets, where no vertices in the same set have an edge between them.



- The maximum bipartite matching problem, given a bipartite graph, is to choose the largest possible set of edges in that graph such that no one vertex is incident to more than one chosen edge.
- There is a clear flow formulation for this problem.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Modify the original bipartite graph by making each edge a directed edge from the first set to the second set, with capacity 1.
- Attach an edge of capacity 1 from s to every vertex in the first set, and an edge of capacity 1 from every vertex in the second set to t .

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

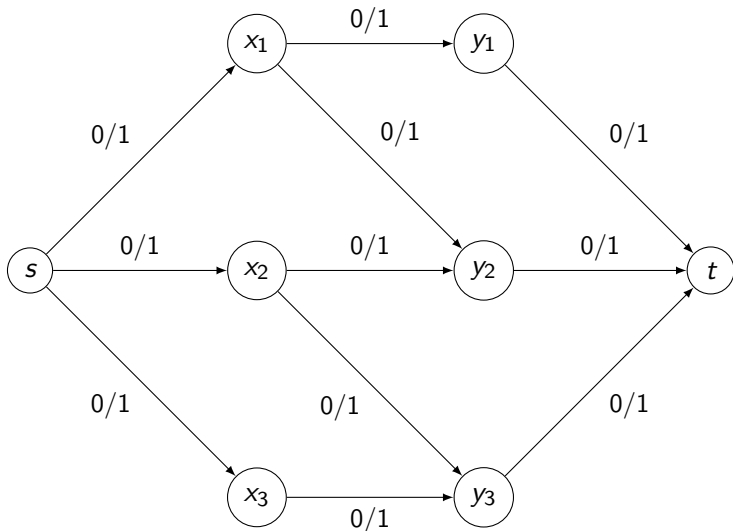
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- The size of the largest matching is the maximum flow in this graph.
- Since the flow of this graph is at most V , Ford-Fulkerson and Edmonds-Karp run in $O(VE)$.
- Dinic's algorithm is even faster!

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

Definition

A unit graph is one where every non-source, non-sink node, say u , satisfies at least one of the following:

- u has only one incoming edge, whose capacity is 1
- u has only one outgoing edge, whose capacity is 1.

In other words, no vertex appears in more than one flow path.

Fact

Dinic's algorithm runs in $O(E\sqrt{V})$ on unit graphs.

Fact

The flow network for maximum bipartite matching is a unit graph.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Aside: Augmenting paths look very nice in bipartite graphs. There's the matched component and unmatched vertices and augmenting paths just alternate between following unmatched edges and matched edges.
- Also on bipartite graphs we get nice theorems like Hall's Marriage Theorem.
- For these reasons, we can say a lot more about the structure of bipartite graphs. Also certain problems on bipartite graphs are a lot easier than on general graphs.
- But won't get time to see this in this course.

Network Flow

Flow Networks

Maximum
Flow

Flow
Algorithms

Ford-
Fulkerson

Edmonds-
Karp

Dinic

Faster
Algorithms

Bipartite
Matching

Min Cut

Related
Problems

Example
Problems

- 1 Flow Networks
- 2 Maximum Flow
- 3 Flow Algorithms
 - Ford-Fulkerson
 - Edmonds-Karp
 - Dinic
 - Faster Algorithms
- 4 Bipartite Matching
- 5 Min Cut**
- 6 Related Problems
- 7 Example Problems

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- The dual to max flow is min cut.
- This is interesting in its own right! Frequently this shows up in unexpected places.
- Helps solve some optimal assignment problems.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- To get the value of the minimum cut, just run max flow.
- To extract the actual edges in the minimum cut, we use the fact that all of them must be saturated.
- We do a graph traversal starting from s and only traverse edges that have positive remaining capacity and record which vertices we visit.
- The edges which have a visited vertex on one end and an unvisited vertex on the other will form the minimum cut.
 - Compare to the definition of a s - t cut
- Make sure you only count actual edges, and not virtual edges!

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

```
void check_reach(int u, vector<bool>& seen) {
    if (seen[u]) { return; }
    seen[u] = true;
    for (int v : adjList[u]) {
        if (adjMat[u][v] > 0) { check_reach(v, seen); }
    }
}

vector<pair<int,int>> min_cut(int s, int t) {
    ll value = dinic(s,t);

    vector<bool> seen(n,false);
    check_reach(s, seen);

    vector<pair<int,int>> ans;
    for (int u = 0; u < n; u++) {
        if (!seen[u]) { continue; }
        for (int v : adjList[u]) {
            if (!seen[v] && !is_virtual[u][v]) { // need to record this in add_edge()
                ans.emplace_back(u,v);
            }
        }
    }
    return ans;
}
```

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- One of the canonical examples of an assignment problem solvable by flow.
- **Problem Statement:** There are n projects and m machines.
Each machine has a cost c_j . Each project has a revenue r_i .
Each project depends on a set of machines but machines can be shared among projects.
Determine a set of projects and machines that maximizes profit.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Input Format:** First line contains 3 integers n, m ($1 \leq n, m \leq 1000$), the numbers of projects and machines. Next line contains n integers, the revenues (non-negative). After that is a line with m integers, the costs (non-negative). Next line contains one integer, d ($0 \leq d \leq 1000$), the number of dependencies. The following d lines each contain a pair i, j ($1 \leq i \leq n, 1 \leq j \leq m$), denoting that project i is dependent on machine j .

Network Flow

Flow Networks

Maximum
Flow

Flow
Algorithms

Ford-
Fulkerson

Edmonds-
Karp

Dinic

Faster
Algorithms

Bipartite
Matching

Min Cut

Related
Problems

Example
Problems

- **Sample Input:**

```
3 3
100 200 150
200 100 50
4
1 1
1 2
2 2
3 3
```

- **Sample Output:**

```
200
```

- **Explanation:** The optimal choice is to purchase machines 2 and 3 and select projects 2 and 3.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Here we have a classical optimization problem with constraints. Let's make these explicit.
- Let P be the projects selected and Q the machines purchased. We are maximizing

$$\sum_{i \in P} r_i - \sum_{j \in Q} c_j.$$

- **Useful trick:** Because we want to *minimize* cost, we want to rewrite our problem as a minimization. So let P^c be the set of projects *not* selected. Then we need to minimize

$$\sum_{i \in P^c} r_i + \sum_{j \in Q} c_j.$$

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Our constraints are, for all dependences i, j ,
 $i \in P \implies j \in Q$. In terms of P^c , this can be rephrased as

$$i \in P^c \vee j \in Q.$$

- We can formulate this as a min cut! We will have edges representing projects and machines, with costs r_i and c_j respectively. Cutting an edge represents adding i to P^c or j to Q .
- To formulate our constraints, for each dependence i, j we want a path from source to sink using the edge representing project i and the edge representing machine j .

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Formal Construction:** We will form a bipartite graph. One bipartition represents projects, the other machines.
- Projects are connected to the source with edges representing their revenues.
- Machines are connected to the sink with edges representing their costs.
- **Useful Trick:** An edge with capacity infinity connects project i to machine j if i is dependent on j .
- The implementation is just constructing the flow network then calling your flow code.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

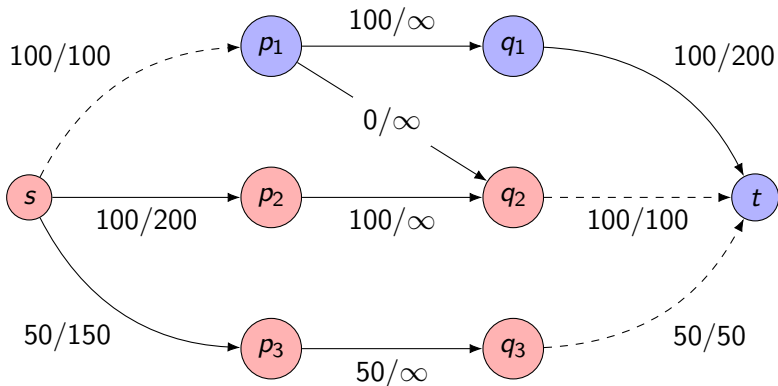
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

```
// Construction:
// Nodes 0 to n-1 are the project nodes.
// Nodes n to n+m-1 are the machine nodes.
// Node n+m is the source, n+m+1 is the sink.

int main(void) {
    int n, m, d;
    cin >> n >> m;
    FlowNetwork net(n+m+2);
    int s = n+m, t = n+m+1;
    int totalRevenue = 0;
    for (int i = 0; i < n; i++) {
        int r;
        cin >> r;
        net.add_edge(s, i, r);
        totalRevenue += r;
    }
    for (int j = 0; j < m; j++) {
        int c;
        cin >> c;
        net.add_edge(n+j, t, c);
    }
    cin >> d;
    for (int k = 0; k < d; k++) {
        int a, b;
        cin >> a >> b;
        net.add_edge(a-1, n+b-1, INF); // INF > all possible outputs.
    }
    cout << totalRevenue - net.dinic(s,t) << '\n';
}
```

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Summary:** It's a classic example but it shows the general gist of how many min cut problems work.
- First we rephrase our optimization problem into a minimization problem. Often we have to negate variables for this. We want cutting an edge to represent something with a cost.
- Next we try to phrase the constraints as cuts. Here it's important to have a good grasp on what the edges are, we are aiming to form paths from source to sink so that our constraints translate are equivalent to all paths having a cut edge.
- There are useful constructions that appear repeatedly. Adding infinite weighted edges is common.
- Often we also hope we can reduce the problem to something similar to the bipartite example here.

Network Flow

Flow Networks

Maximum
Flow

Flow
Algorithms

Ford-
Fulkerson

Edmonds-
Karp

Dinic

Faster
Algorithms

Bipartite
Matching

Min Cut

Related
Problems

Example
Problems

- 1 Flow Networks
- 2 Maximum Flow
- 3 Flow Algorithms
 - Ford-Fulkerson
 - Edmonds-Karp
 - Dinic
 - Faster Algorithms
- 4 Bipartite Matching
- 5 Min Cut
- 6 Related Problems**
- 7 Example Problems

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- A graph has vertex capacities if there are also capacity restrictions on how much flow can go through a vertex.
- This is solved by splitting each vertex into two vertices, an “in” vertex and an “out” vertex.
- For some vertex u with capacity c_u , we add an edge from u_{in} to u_{out} with capacity c_u .
- Incoming edges go to u_{in} and outgoing edges leave from u_{out} with their original capacities.
- Useful for flow problems where vertices have restrictions.
- Also useful for cut problems where costs are associated with vertices.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

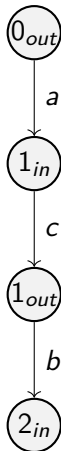
Related Problems

Example Problems

Original graph:



Node 1 has capacity c :



Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Sometimes a problem will naturally have multiple sources or sinks.
- Just make all the sources into regular nodes and connect them with infinite edges to a “supersource”, the actual source of the flow graph.
- Same with sinks.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

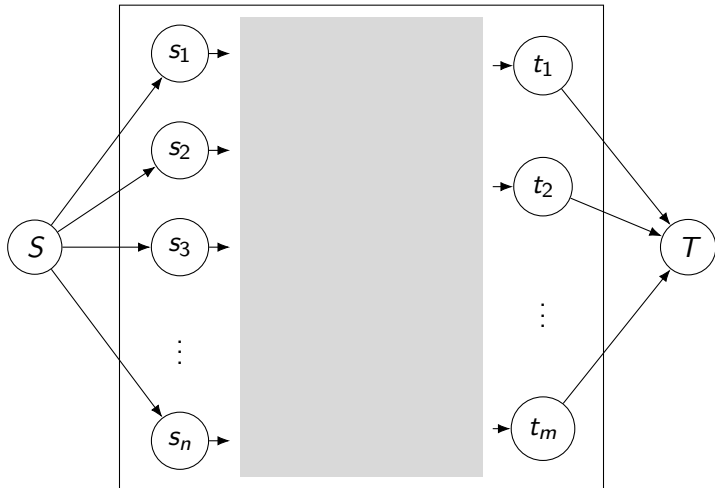
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Sometimes you want to find flow or min cut in an undirected graph.
- Just duplicate each edge, one going forwards, one going backwards.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- A set of paths is edge-disjoint if no two paths use the same edge.
- To find the maximum number of edge-disjoint paths from s to t , make a flow graph where all edges have capacity 1.
- The maximum flow of this graph will give the answer.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- A vertex cover in a graph is a set of vertices which touches at least one endpoint of every edge.
- By König's theorem, the size of the maximum matching is equal to the number of vertices in a minimum vertex cover.
- Actually this is just min-cut max-flow. Using our earlier construction of a flow network for bipartite graphs, the min cut corresponds to a max vertex cover.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

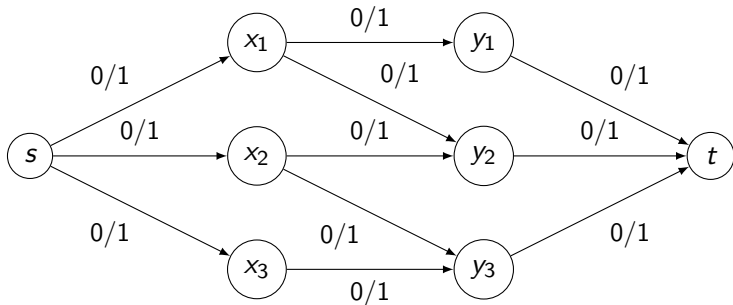
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



- Edges directly connected to source and sink represent vertices. If these are cut, the corresponding vertex is in the min vertex cover.
- If the cut contains the intermediary edges, pick either endpoint to be in the min vertex cover.

Network Flow

Flow Networks

Maximum
Flow

Flow
Algorithms

Ford-
Fulkerson

Edmonds-
Karp

Dinic

Faster
Algorithms

Bipartite
Matching

Min Cut

Related
Problems

Example
Problems

- 1 Flow Networks
- 2 Maximum Flow
- 3 Flow Algorithms
 - Ford-Fulkerson
 - Edmonds-Karp
 - Dinic
 - Faster Algorithms
- 4 Bipartite Matching
- 5 Min Cut
- 6 Related Problems
- 7 Example Problems

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- We just covered the basic theory of flow.
- But the truly remarkable part of flow (and especially min cut) is it appears in problems you don't expect it to!
- To me it is because flow and min cut try to solve a very general problem, either assignment or optimization under constraints.
- Both in full generality are NP-hard and encompass most problems we care about. So even being able to solve a small subclass of these is pretty remarkable.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- There are 2 main parts to flow problems: recognizing they are flow and constructing the right flow graph.
- The former is difficult, but you get some intuition after a while. Flow problems tend to be unsolvable by anything except flow.
- I feel a decent approach to start with is, if the problem requires assignments with no obvious greedies then flow is worth considering. If the problem requires optimization with constraints with no obvious greedy, consider min cut.
- Constructing the right flow graph requires a lot of practice but there are common reusable constructions.
- I will try to show some useful basic examples and things to keep in mind.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Problem Statement:** Disneyland has n visitors and m attractions. Each visitor is interested in a set of attractions and will be happy if they can ride at least one. Each attraction has a limit on the number of visitors that can ride.

In addition, certain visitors are entitled to enter during magic hours. Only some of the attractions are open during magic hours. The normal limits don't apply but at most k visitors can enter during magic hours.

What is the maximum number of visitors that can be made happy?

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Input Format:** First line 3 integers, n, m, k .

$1 \leq n, m, k \leq 1000$.

Next line contains m integers, the limits for each of the attractions.

Next line contains m integers. The i th is 1 iff attraction i is open during magic hours, or 0 otherwise.

The next n lines each describe a visitor. Each line is of the form $b \ C \ x_1 \ \dots \ x_C$ where

- b is 1 if this visitor is eligible for magic hours, or 0 otherwise.
- C is the number of attractions the visitor is interested in and
- $\{x_1, \dots, x_C\}$ is the set of these attractions, 1-indexed.

There are at most 2000 visitor-attraction pairs.

Network Flow

Flow Networks

Maximum
Flow

Flow
Algorithms

Ford-
Fulkerson

Edmonds-
Karp

Dinic

Faster
Algorithms

Bipartite
Matching

Min Cut

Related
Problems

Example
Problems

- **Sample Input:**

3 2 1

1 1

1 0

1 1 1

0 2 1 2

0 1 1

- **Sample Output:** 3

- **Explanation:** Visitor 2 rides attraction 2 and visitor 3 rides attraction 1. Visitor 1 rides attraction 1 during magic hours.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Seems like an assignment problem with no obvious greedy (e.g: try $k = 0$). So flow seems suitable here.
- The task now is to translate the requirements into a flow graph.
- And until you become familiar, this step will likely be a lot of trial and error.
- So I'm just going to iterate through some examples.
- I think it is also important to be able to determine if a flow graph is correct, so expect some wrong flow graphs!

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Let's start with basics. What happens if there are no magic hours?
- Question is just bipartite matching with constraints on attractions.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

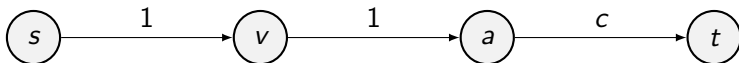
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



Here, c is the capacity of the attraction.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- So far so good. How to handle magic hours?
- **Try 1:** Create a copy of the graph for magic hours. This graph only contains visitors that are entitled to magic hours and rides open during magic hours.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

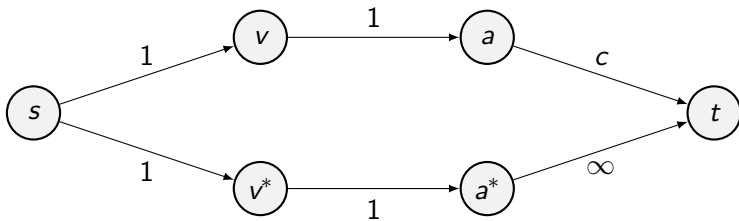
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



Assume visitor v and attraction a are both active during magic hours.

What's wrong?

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- An obvious problem. We aren't accounting for k , the limit on visitors during magic hours.
- There's a standard fix here, add an extra source to the magic hours side of the graph with capacity k .

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

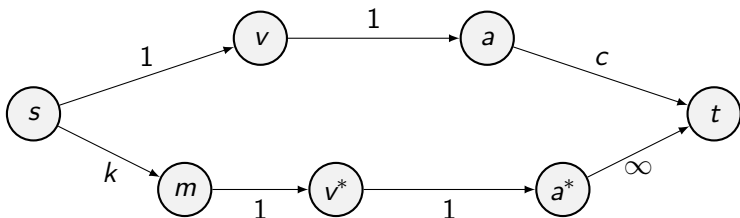
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



What's wrong?

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- The same visitor might be double counted; they might visit an attraction during the day and we would count them again during magic hours.
- This is a common mistake with flow graphs, you try to add in a new constraint and in the process you invalidate an old constraint.
- So we need a single bottleneck per visitor.
- We can try to pass the magic hours source through the visitors.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

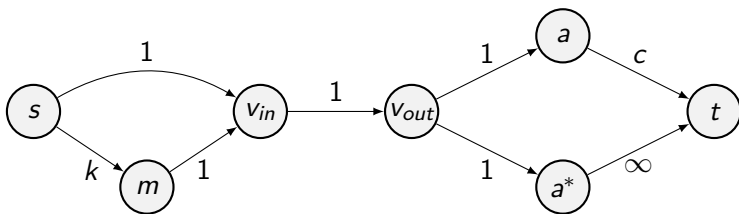
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



What's wrong?

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Many things. The magic hours source doesn't actually *do* anything. More concretely, we are allowing the visitors visiting during magic hours to visit all attractions, not just the magic hours attractions.
- Mixing up different paths is another very common mistake with flow graphs.
- Okay, let's try putting the magic hours constraint after the constraint on the visitors.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

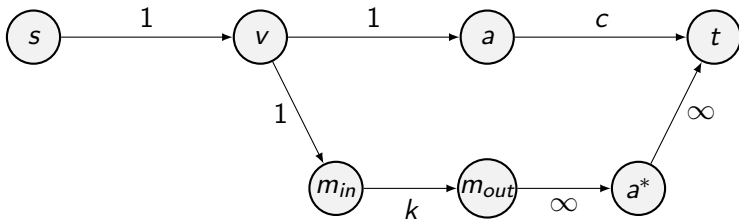
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



Note we split the magic hours node into 2 nodes since we have a constraint on the node itself.

What's wrong?

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Not obvious when there's only 1 visitor. But with many, by passing through the node for magic hours, you're allowing them to visit ANY of the attractions open during magic hours, not just the ones they are interested in seeing. How to fix?
- Note we don't actually care which attraction a visitor visits during magic hours, just that there is at least one attraction they are interested in and is open during magic hours.
- So precalculate which visitors can actually visit an attraction during magic hours.
- After that, we don't care about the attractions connected to the magic hours node. So we can connect the magic hours node directly to the sink.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

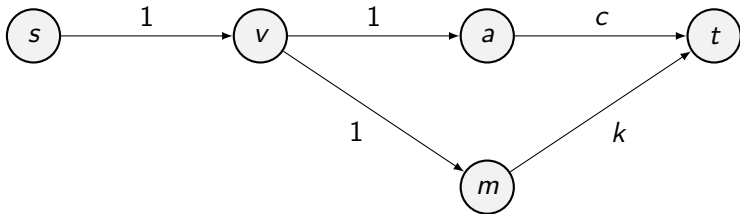
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

```
int main(void) {
    int n, m, k;
    cin >> n >> m >> k;
    int magic = n+m, s = n+m+1, t = n+m+2;
    FlowNetwork net(n+m+3);

    for (int j = 0; j < m; j++) {
        int c;
        cin >> c;
        net.add_edge(n+j, t, c); // attractions are nodes n to n+m-1
    }
    vector<int> magicOpen(m);
    for (int j = 0; j < m; j++) { cin >> magicOpen[j]; }

    for (int i = 0; i < n; i++) {
        net.add_edge(s, i, 1); // visitors are nodes 0 to n-1
        int b, C, x;
        cin >> b >> C;
        for (int j = 0; j < C; j++) {
            cin >> x;
            x--;
            add_edge(i, n+x, 1);
            if (b && magicOpen[x]) {
                net.add_edge(i, magic, 1);
                b = 0; // avoid repetition, not really necessary
            }
        }
    }

    net.add_edge(magic, t, k);
    cout << net.dinic(s,t) << '\n';
}
```

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Complexity?** $O(n + m)$ vertices, $O(n + m + a)$ edges where $a \leq 2000$ is the number of visitor-attraction pairs. So about 2000 vertices and 5000 edges.
- Dinic's is $O(V^2E)$ which directly evaluated gives about 16 billion (which is very likely good enough).
- However, the max flow is at most n (the number of visitors), so any augmenting path algorithm (FF, EK or Dinic) will run in $O(Ef)$ which evaluates to just 5 million.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Keep in mind these traps!
- Whenever you think you have a flow network it should be because you think each plausible flow path means something. You should check:
 - every possible choice is represented by some flow path
 - every flow path corresponds to some valid choice.
- It would be a good idea to double check you understand why every edge has the capacity it has in this example.
- Only real method to learn is probably practice. When I'm doing flow, I still find myself spending most of my time iterating through flow graphs.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Problem Statement:** My garden is a $n \times m$ grid, some squares containing flowers. As I learnt recently, flowers need water ...
To water them, I can purchase some line sprinklers. Each line sprinkler can water a single row or column. What is the minimum number of line sprinklers I need?
- **Input Format:** The first line contains 2 integers, n, m , $1 \leq n, m \leq 1000$. The following n lines each contain m bits. A 1 denotes a square with a flower.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Sample Input:**

3 3

001

111

001

- **Sample Output: 2**

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

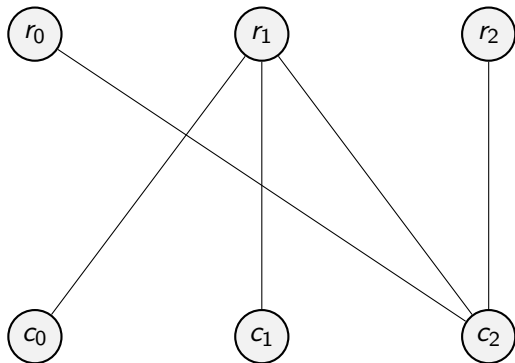
Related Problems

Example Problems

- What are our constraints here?
- We have one for each flower. Each flower says we need to pick its row or its column (maybe both). In our flow formulation, it would make sense that each flower is an edge.
- So our nodes represent the rows and columns.
- **Key Observation:** This graph is naturally bipartite. If our flowers are edges, they can only connect a row node to a column node.

Network Flow

Sample Input again:



Flow Networks

Maximum
Flow

Flow
Algorithms

Ford-
Fulkerson

Edmonds-
Karp

Dinic

Faster
Algorithms

Bipartite
Matching

Min Cut

Related
Problems

Example
Problems

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- What are we being asked for?
- Minimum set of vertices adjacent to at least one edge.
- Min vertex cover!
- We could have also solved the flow directly without realising this. Noting it is a bipartite graph is key however.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

```
// Construction:
// Nodes 0 to n-1 are the row vertices.
// Nodes n to n+m-1 are the column vertices.
// Node n+m is the source, n+m+1 is the sink.

int n, m;

int main(void) {
    cin >> n >> m;
    int s = n+m, t = n+m+1;
    FlowNetwork net(n+m+2);

    for (int r = 0; r < n; r++) {
        for (int c = 0; c < m; c++) {
            char ch;
            cin >> ch;
            if (ch == '1') { net.add_edge(r, c+n, 1); }
        }
    }

    for (int r = 0; r < n; r++) { net.add_edge(s, r, 1); }
    for (int c = 0; c < m; c++) { net.add_edge(c+n, t, 1); }

    cout << net.dinic(s,t) << '\n';
}
```

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Complexity?** $O(n + m)$ vertices, $O(nm)$ edges, with Dinic's it is $O(nm\sqrt{n + m})$, about 3×10^7 . More than fast enough, especially for a flow problem.
- How could you get the feeling this might involve flow?
- You're roughly being asked to minimize a value, subject to some constraints.
- If you play with it a bit, you'll realize a greedy solution is hard. So the next natural thing to try is flow.
- Also the graph is very naturally bipartite.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- It is useful to be able to recognize if a problem involves a bipartite graph quickly.
- A sometimes useful characterization: Bipartite graphs are graphs with no odd cycles.
- Also a graph is bipartite iff it can be 2-colored. This allows you to check bipartiteness in $O(V + E)$ with a DFS.
- Common examples:
 - Any time you naturally have two different kinds of nodes.
 - On grids, 4-adjacency forms a bipartite graph. Why?
 - For same reason, so does knight jumps!
 - The rows and columns of a grid.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Problem statement** Freddy Frog is on the left bank of a river. He has to go to the right bank to collect a gift, then return to his home on the left bank. The distance between the left and the right bank is D ($1 \leq D \leq 10^9$) metres. Unfortunately, he cannot touch the polluted water as it is extremely contaminated.

However, there are n ($1 \leq n \leq 100$) rocks floating on the river. There are rocks of two sizes: the bigger ones can withstand any weight but the smaller ones start to drown as soon as any mass is placed on it. He can land on every small rock at most one time, but can use the bigger ones as many times as he likes.

Freddy's journey will consist of a series of jumps. Can you find a path that minimizes the longest jump that Freddy will perform?

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- 2 constraints here, we both have a constraint on maximum distance and constraints on paths.
- Want to shave a constraint, it will be annoying to have to deal with both at the same time. How?
- **Binary search!**
- We can binary search for the smallest maximum edge weight. So from now on we have a decision problem:
- Given we can jump up to X distance, is there a path there and back for Freddy Frog?

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- We need to find a path from our source to our sink, and then back again, that does not cross any of the “small” vertices more than once.
- Keeping track of the state of the vertices after going to the sink and then coming back seems difficult.
- **Observation 1:** We can transform it into the equivalent problem of finding two paths from the source to the sink.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- To find two vertex disjoint paths from the source to the sink, we need to find a flow of at least 2 in a flow graph with vertex capacities 1. Edges with infinite capacities connect 2 stones that are within distance X .
- But this formulation will only allow us to visit any of the “big” vertices once.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- The only thing that enforces that restriction in our graph is the vertex capacity of 1 in our original construction.
- So if we set the vertex capacity for every “big” vertex to be infinite, then we have a solution.
- As always, pick infinite to be larger than any necessary finite capacity.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- We need $O(\log DT)$ (where T depends on required precision) iterations of our binary search to find this maximum edge weight. We can do a max-flow computation in $O(Ef) = O(n^2)$, because we can terminate the max-flow once f hits 2.
- The total runtime is $O(n^2 \log DT)$.
- **Implementation**

```
double lo = 0, hi = 1e10;
for (int it = 0; it < 70; it++) {
    double mid = (lo + hi) / 2;
    if (cando(mid)) { hi = mid; }
    else { lo = mid; }
}
cout << lo << '\n';
```

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

```
inline double sqdist(double x1, double y1, double x2, double y2) {  
    return (x1-x2)*(x1-x2) + (y1-y2)*(y1-y2);  
}  
  
bool cando(double X) { // allow jumps of up to X  
    if (D < X) { return true; }  
    int s = 2*n, t = 2*n+1, tt = 2*n+2; // source, fake sink, real sink  
    FlowNetwork net(2*n+3);  
  
    for (int i = 0; i < n; i++) {  
        int cap = isbig[i] ? INF : 1;  
        net.add_edge(2*i, 2*i+1, cap); // in to out  
    }  
  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            if (sqdist(x[i], y[i], x[j], y[j]) < X*X) {  
                net.add_edge(2*i+1, 2*j, INF); // out to in  
            }  
        }  
    }  
  
    for (int i = 0; i < n; i++) {  
        if (x[i] < X) { net.add_edge(s, 2*i, INF); }  
        if (x[i] + X > D) { net.add_edge(2*i+1, t, INF); }  
    }  
  
    net.add_edge(t, tt, 2); // limit max flow to 2  
    return net.dinic(s,tt) == 2;  
}
```

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Problem Statement:** An image is made of n pixels. Each can be either in the background or foreground. For the i th pixel, you get f_i points if it is in the foreground, b_i points if it is in the background. Furthermore, there are m pairs of pixels that you would prefer to have the same assignment. For the k th pair, you pay a penalty of p_k if pixel a_k has a different assignment to pixel b_k .
Maximize points less penalties.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Input Format:** First line, 2 integers n and m ($1 \leq n, m \leq 1000$).
Next line contains n integers, the values f_i .
Next line contains n integers, the values b_i .
Next m lines each contain three integers, a_k , b_k and p_k , describing one of the penalty pairs.

Network Flow

Flow Networks

Maximum

Flow

Flow

Algorithms

Ford-

Fulkerson

Edmonds-

Karp

Dinic

Faster

Algorithms

Bipartite

Matching

Min Cut

Related

Problems

Example

Problems

• Sample Input:

3 2

5 1 3

1 5 2

1 2 1

2 3 5

• Sample Output: 11

- **Explanation:** Assign the pixels as (foreground, background, background). You pay a penalty of 1 because the first 2 pixels have different assignments.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Optimization problem with constraints and no obvious greedy. Reasonable to try min cut.
- We need to change this into a minimization. Write out what we are maximizing.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Let F be the set of pixels in the foreground and B be background. Say that $a_i = 1$ iff pixel i is in the foreground.
- Let p_{ij} be the penalty if pixel i has a different assignment to pixel j . This is 0 if there is no constraint involving pixels i and j

- Then we are maximizing

$$\sum_{i \in F} f_i + \sum_{i \in B} b_i - \sum_{a_i \neq a_j} p_{ij}.$$

- This is equivalent to minimizing (S^c is the complement of S):

$$\sum_{i \in F^c} f_i + \sum_{i \in B^c} b_i + \sum_{a_i \neq a_j} p_{ij}.$$

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



$$\sum_{i \in F^c} f_i + \sum_{i \in B^c} b_i + \sum_{a_i \neq a_j} p_{ij}$$

- This suggests what some of our edges should be and what cutting edges should mean.
- For each pixel i , we need to have an edge representing pixel i is in the foreground.
- It should have cost f_i and cutting it represents that pixel i is not in the foreground.
- Same with an edge representing the background.
- Call the first edge $fore_i$ and the second edge $back_i$.
- We need an edge, say e_{ij} representing each penalty cost too. Cutting this edge represents that pixels i and j have different assignments.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- What are our constraints?
- For each of the m penalties,

$$(\text{cut } e_{ij}) \vee (i \in F \iff j \in F).$$

- We can unroll this into two constraints involving just 'or' statements:

$$\begin{aligned} &(\text{cut } e_{ij}) \vee (i \in F^c) \vee (j \in B^c) \\ &(\text{cut } e_{ij}) \vee (i \in B^c) \vee (j \in F^c). \end{aligned}$$

- Since we are defining $(i \in F^c)$ to be the same as cutting fore_i , we can rewrite the first condition as

$$(\text{cut } e_{ij}) \vee (\text{cut } \text{fore}_i) \vee (\text{cut } \text{back}_j).$$

- This suggests what our paths should be.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

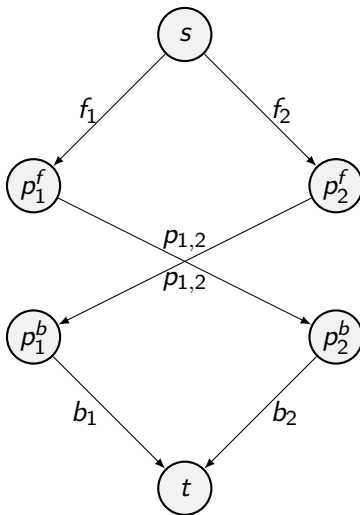
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



What's wrong?

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Not every cut corresponds to a meaningful assignment.
- There's no guarantee we cut either $fore_1$ or $back_1$. If we don't, this means pixel 1 is in both the foreground and background!
- So we can try to fix this by making a path that includes both $fore_1$ and $back_1$. Here we apply the standard trick of introducing an ∞ weighted edge.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

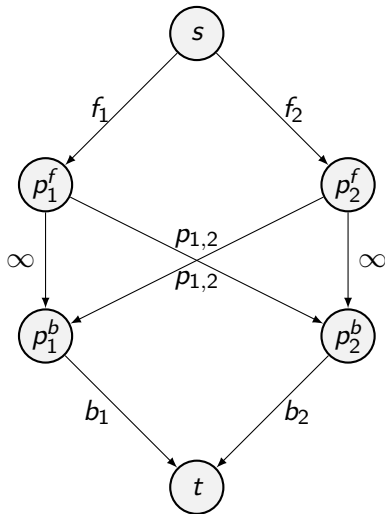
Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems



What's wrong?

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford-Fulkerson

Edmonds-Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- Now at least one of $fore_1$ or $back_1$ is cut.
- But we might cut both these edges!
- This corresponds to having pixel 1 in neither the foreground nor background! Less obviously an issue but one can come up with a breaking case.
- There are two solutions to this problem, each showcasing a worthwhile construction.

Network Flow

Flow Networks

Maximum
FlowFlow
AlgorithmsFord-
FulkersonEdmonds-
Karp

Dinic

Faster
AlgorithmsBipartite
Matching

Min Cut

Related
ProblemsExample
Problems

- The first is: to fix this construction, we need to enforce we cut exactly one of $fore_i$ or $back_i$. Enforcing “exactly one” constraints is hard with min cut in general.
- Instead, we will force the constraint that we cut exactly n of the edges in

$$\{fore_i\}_{i=1}^n \cup \{back_i\}_{i=1}^n.$$

- Then since the earlier construction forces us to cut at least one of $fore_i$ or $back_i$, this is equivalent to cutting exactly one of the 2 for each i .
- We enforce this by artificially inflating the flows of the edges $fore_i$ and $back_i$ so that no optimal solution would cut more than the bare minimum necessary, which is n .

Network Flow

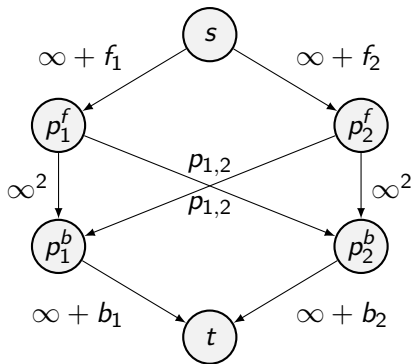
Flow Networks

Maximum
FlowFlow
AlgorithmsFord-
FulkersonEdmonds-
Karp

Dinic

Faster
AlgorithmsBipartite
Matching

Min Cut

Related
ProblemsExample
Problems

- Pick ∞ so that $\infty > \sum_{i,j} p_{i,j}$.
- Pick ∞^2 so that $\infty^2 > n \cdot \infty + \sum_i f_i$.

Network Flow

Flow Networks

Maximum
FlowFlow
AlgorithmsFord-
FulkersonEdmonds-
Karp

Dinic

Faster
AlgorithmsBipartite
Matching

Min Cut

Related
ProblemsExample
Problems

- Then, our choices of ∞ guarantee that:
 - we never cut more than n edges with an ∞ (it's better to just cut all the $p_{i,j}$ edges instead) and
 - we never cut an edge with an ∞^2 (it's better to just cut all the $\infty + f_i$ edges).
- Our min cut will be of the form $n \cdot \infty + A$ where A is the actual answer.
- Worth noting: we introduced 2 edges for each $p_{i,j}$. This should give you pause but is okay here because no min cut will cut both.
- **Exercise:** Implement this.

Network Flow

Flow Networks

Maximum
FlowFlow
AlgorithmsFord-
FulkersonEdmonds-
Karp

Dinic

Faster
AlgorithmsBipartite
Matching

Min Cut

Related
ProblemsExample
Problems

- The alternative solution uses a slightly different construction and thinks of cutting as more a way of partitioning nodes.
 - Compare to definition of a s - t cut.
- It turns out the problem here is actually that we separate each pixel into 2 nodes.
- If we instead don't, everything just happens to work out.

Network Flow

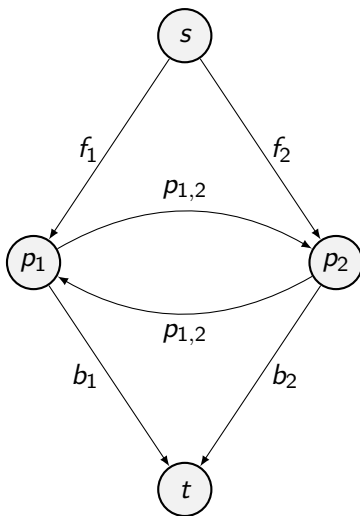
Flow Networks

Maximum
FlowFlow
AlgorithmsFord-
FulkersonEdmonds-
Karp

Dinic

Faster
AlgorithmsBipartite
Matching

Min Cut

Related
ProblemsExample
Problems

Network Flow

Flow Networks

Maximum
FlowFlow
AlgorithmsFord-
FulkersonEdmonds-
Karp

Dinic

Faster
AlgorithmsBipartite
Matching

Min Cut

Related
ProblemsExample
Problems

- This works because an s - t cut, every node is either on the 'source side' or the 'sink side'.
- If the node representing pixel i is on the 'source side', then:
 - the edge $back_i$ crosses the partition, so we count b_i towards the capacity of the cut, but
 - $fore_i$ does not cross the partition, so it does not contribute towards the capacity of the cut.

We take this to represent a pixel in the foreground.

- The opposite holds for a pixel whose vertex is on the 'sink side'. Therefore every pixel belongs to exactly one of the layers.

Network Flow

Flow Networks

Maximum
FlowFlow
AlgorithmsFord-
FulkersonEdmonds-
Karp

Dinic

Faster
AlgorithmsBipartite
Matching

Min Cut

Related
ProblemsExample
Problems

- Finally, we need to consider the penalty edges. Which of these are cut?
 - For a pair of pixels i and j in the same layer, neither $p_{i,j}$ edge crosses the partition, so the penalty does not count towards the cut.
 - However, if they are in different layers, exactly one of the two $p_{i,j}$ edges counts. Recall that only edges from the 'source side' to the 'sink side' (and not vice versa) contribute to the capacity of the cut.
- So every s - t cut corresponds to a valid allocation of pixels to layers, and the capacity of that cut is the layer points foregone plus the penalties incurred.
- Min cut solves the problem! **Exercise:** Implement this.

Network Flow

Flow Networks

Maximum Flow

Flow Algorithms

Ford- Fulkerson

Edmonds- Karp

Dinic

Faster Algorithms

Bipartite Matching

Min Cut

Related Problems

Example Problems

- **Complexity?** Both have $O(n)$ vertices and $O(n + m)$ edges, so Dinic's runs in worst case $O(n^2(n + m))$.
- Like Project Selection, Image Segmentation is a useful subproblem to keep in mind.
- It is useful when you need to assign truth values to variables. The main pitfall is not assigning exactly one truth value to each variable and either of the solutions here circumvents this issue.
- Adding infinity to edges is also a generally useful gadget for controlling the number of edges cut. E.g: find the min cut that also cuts the minimum number of edges.